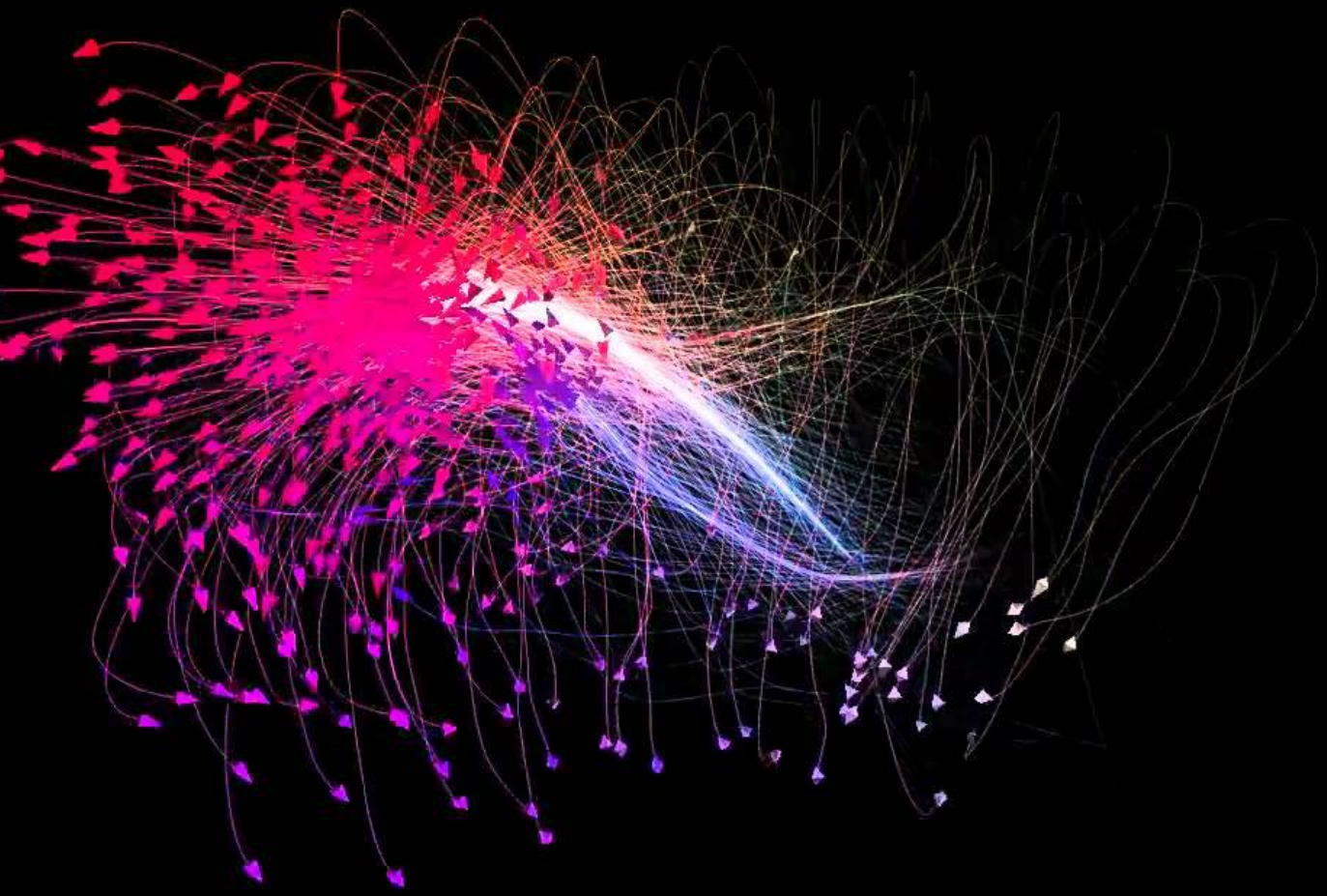
Particle Swarm Optimization



Aston University

Motivation: Population-based Search



Local Search: Recap



- Recall that last week we dealt with:
  - constructive search,
  - perturbative search,
- and that we combined the two to create an optimization algorithm for the TSP.

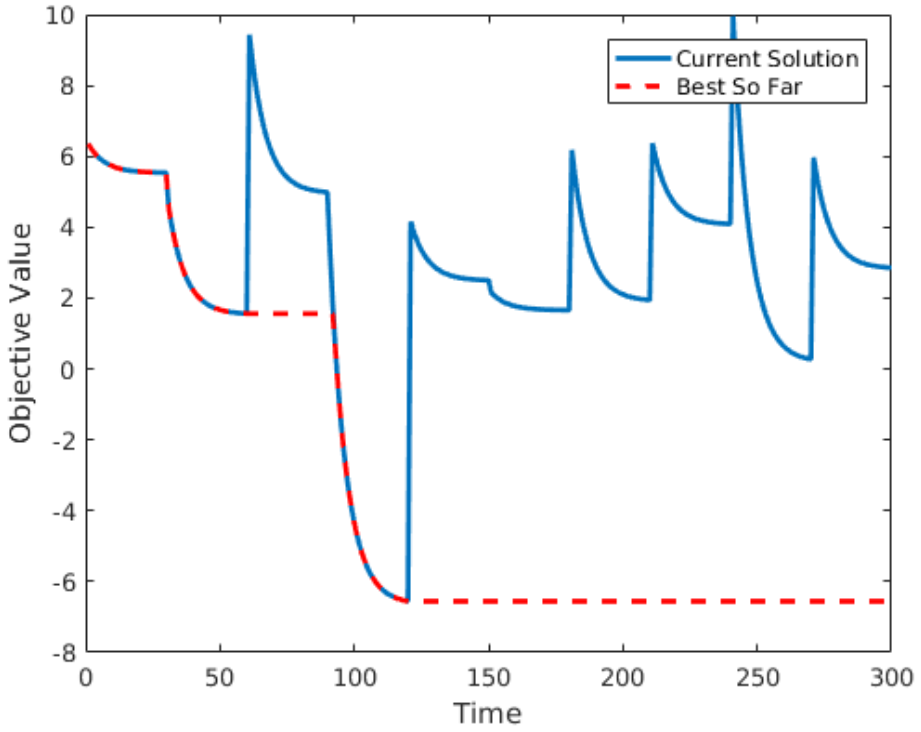
- Any further progress possible?

Key Ideas

- Repeated local search considers each solution candidate **independently**



Iterated Local Search

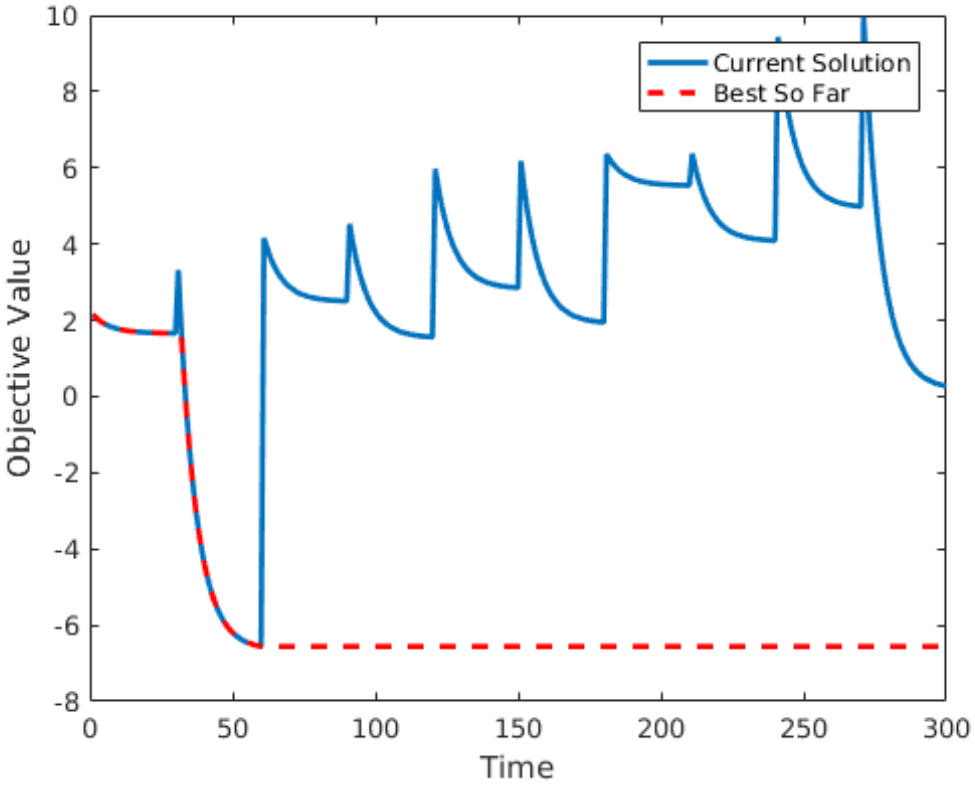


Key Ideas

- Repeated local search considers each solution candidate **independently**
- Can we improve performance by:
  - **prioritising** which solutions to allocate computational resources to?



Sorted Local Search

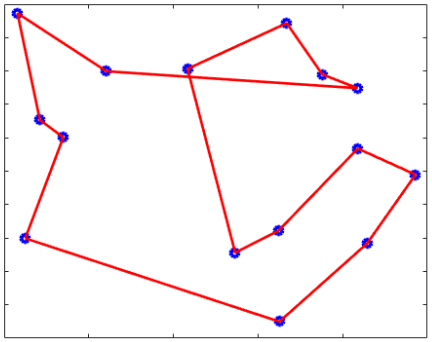


Key Ideas

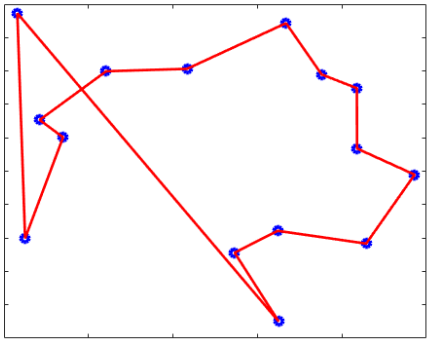
- Repeated local search considers each solution candidate **independently**
- Can we improve performance by:
  - **prioritising** which solutions to allocate computational resources to?
  - **sharing knowledge** between solutions?



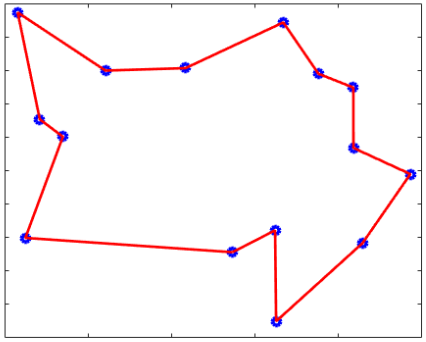
Sharing Solution Components



+



=



?

Key Ideas

- Repeated local search considers each solution candidate **independently**
- Can we improve performance by:
  - **prioritising** which solutions to allocate computational resources to?
  - **sharing knowledge** between solutions?
- This motivates the idea of **population-based search**:
  - start with a set (**population**) of solutions,
  - prioritise exploitation of certain solutions,
  - share knowledge between solutions.



Population-based Search: This Module

- In this module, we will look at two population-based Swarm Intelligence optimization algorithms:
  - **Particle Swarm Optimization** (PSO)
  - **Ant Colony Optimization** (ACO) – independent study

- Both are Computational Intelligence Techniques
  - inspired by natural systems
  - in particular, by swarms

- Later in the module, other population-based optimization algorithms will be introduced



Optimization: Other Considerations

- Based on the local search lecture, we can consider two extreme approaches to optimization:
  - random search
  - Iterated local search with no restart

- Random search is:
  - **slow to improve**, because it doesn't make use of information from previous solutions,
  - **guaranteed to find the best solution eventually**, because it isn't trapped by its prior knowledge.

- Iterated local search is:
  - **quick to improve**, because it makes good use of information from previous solutions,
  - **not guaranteed/likely to find the best solution eventually**, because it can easily get trapped in a local optimum.



Optimization: Other Considerations

- This trade=off between:
  - speed of improvement,
  - chance to find better/best solutions in the long term,
- is known as the **exploration/exploitation**.

- Depending on the requirements of our application, we may want to use an algorithm which prioritises:
  - **exploration** (worse short-term performance, possibility of ongoing improvement)
  - **exploitation** (better short-term performance, less chance of ongoing improvement)

- Population-based search algorithms often give us mechanisms for managing this trade-off.



Introduction



Outline

- At the end of this part of lecture you should…
  - …be able to explain what Swarm Intelligence is and outline some key developments in its history.
  - …be able to describe the ideas behind the development of Particle Swarm Optimization.
  - …understand a given variant of Particle Swarm Optimization well enough to implement it and apply it to real valued optimization problems.



What properties does a **swarm** have?



What is a Swarm?









Good Years for Swarm Intelligence

- *1959* – Pierre-Paul Grassé introduces **Stigmergy** as part of a study of termites.
- *1989* – Gerardo Beni coins term **Swarm Intelligence** as a buzzword for cellular robotics.
- *1986* – Craig Reynolds develops the **Boids** program to simulate bird flocking.
- *1991* – Marco Dorigo publishes **ant system**, the first Ant Colony Optimization (ACO) variant.
- *1995* – Kennedy & Eberhart introduce **Particle Swarm Optimization (PSO)**



What is Swarm Intelligence?

- Original definition: Tied to very specific field – cellular robotics. No longer has this meaning.
- Popular definition: algorithms inspired by (eu)social animal behaviour (esp. **social insects**).
- Swarm Intelligence systems are **distributed problem solvers** composed of:
  - …a **large number** of agents…
  - …each **relatively simple** in relation to the problem…
  - …but can solve the problem through **local interactions**. (decentralised)



PSO: Intuition



Some Videos

Starlings

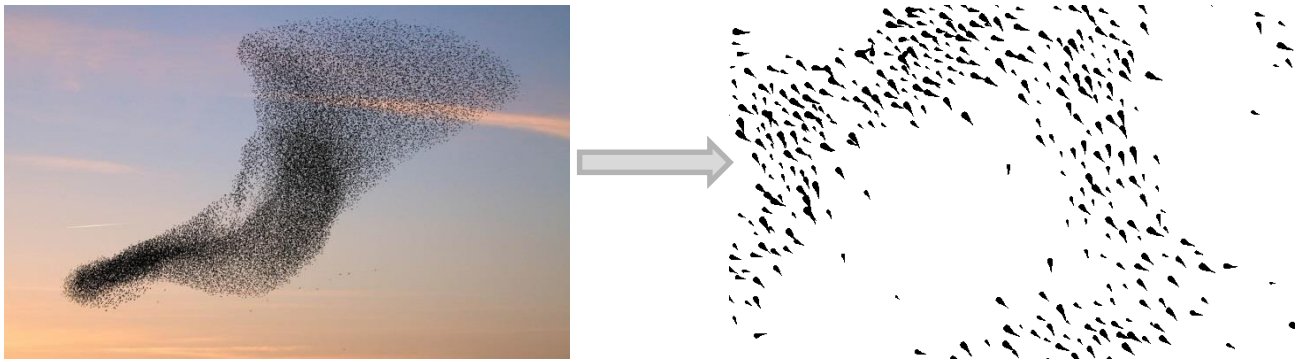
https://www.youtube.com/watch?v=eakKfY5aHmY

Boids

https://www.youtube.com/watch?v=rN8DzlgMt3M



Development: Birds to Boids

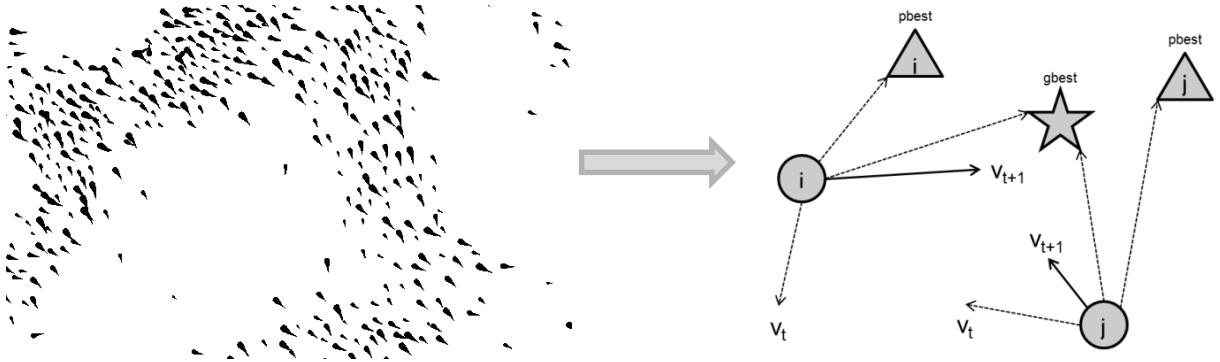


- Reynolds proposed three basic rules to produce realistic flocking behaviour:
  - **Separation** – avoid collisions with neighbours
  - **Alignment** – match neighbours' velocities
  - **Cohesion** – stay close to neighbours
- Three rules result in three **acceleration vectors** which are (typically) averaged
- "**Neighbours**" are (spatially) close individuals

Flocking as a Model of Culture?

- Kennedy (a psychologist) was interested in **culture**.
- Proposed an Adaptive Culture Model in which ideas spread in a population through:
  - **Evaluation** – Individuals tend to rate (the consequences) of their ideas…
  - **Comparison** – …compare their success/failure with their peers'…
  - **Imitation** – …and to copy successful peers.
- Saw parallels with models of bird flocking.

Development: Boids to PSO



- Heppner and Grenander added a “roost” to the boids model.
  - 4th rule: birds attracted to the roost.

- Kennedy and Eberhart asked: **what if birds don’t know the location of the roost**?
  - Instead, they can evaluate their current position to see how good it is.
  - Will the flock find a good roost?

Development: Boids to PSO

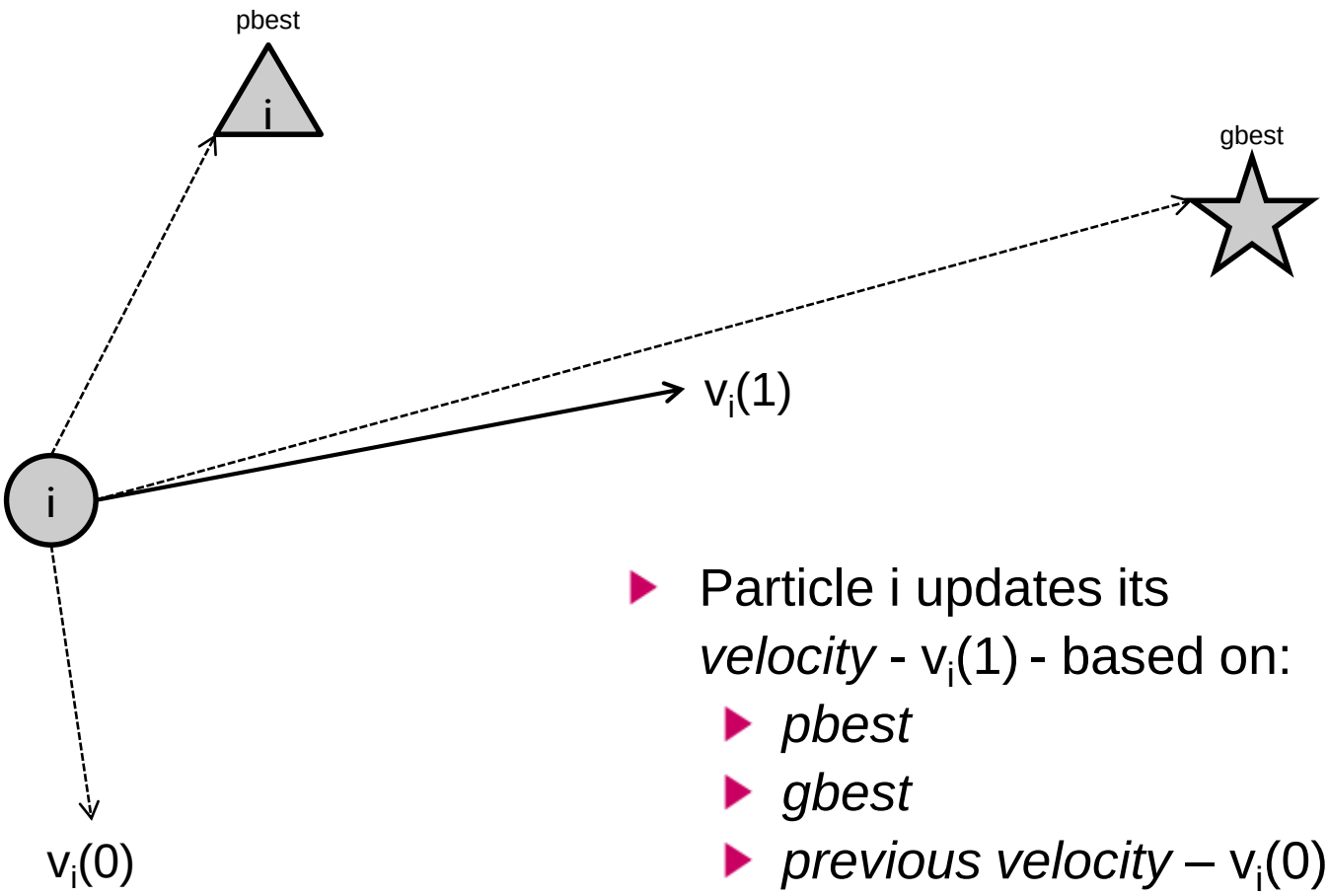
- Kennedy and Eberhart didn't care about visual effect – scrapped the rules of boids

- Used similar framework in which **particles** flying through space:
  - evaluated their own ***positions***
  - remembered a ***personal best position (pbest)***
  - compared their neighbours' personal best positions to find the ***global best position (gbest)***

- Velocity determined by:
  - Inertia – tendency to keep moving in same direction
  - Attraction to personal and global best positions

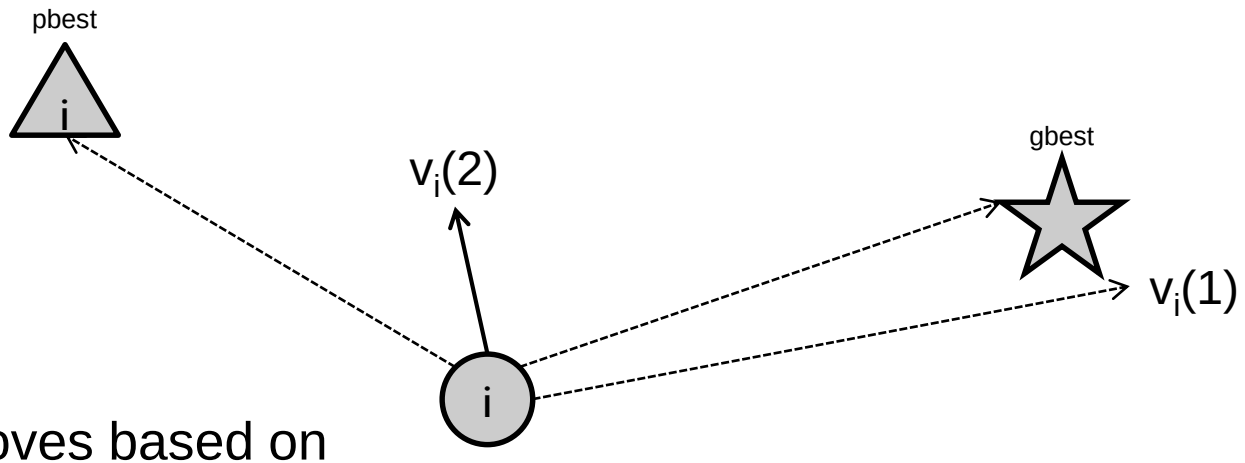


PSO Illustrated



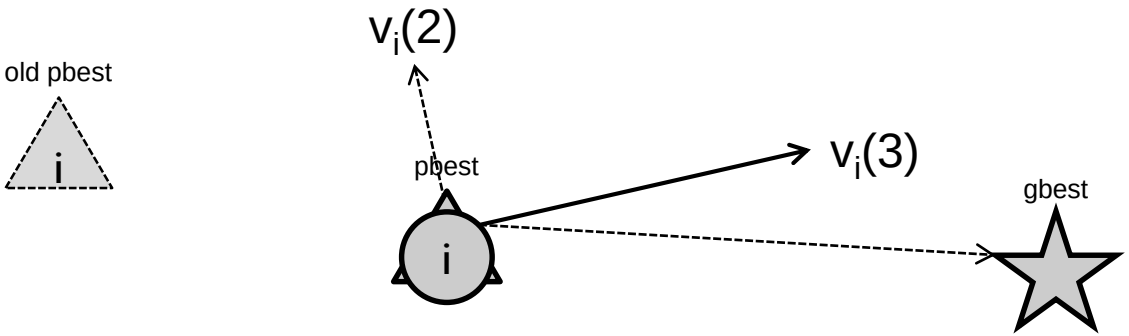
- Particle i updates its *velocity* - $v_i(1)$ - based on:
  - *pbest*
  - *gbest*
  - *previous velocity* – $v_i(0)$

PSO Illustrated



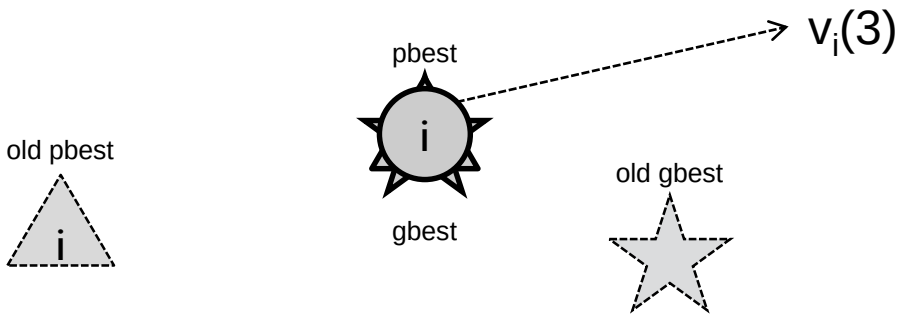
- Particle i moves based on its velocity and evaluates its new position.
- If its new position is worse than *pbest*, then *pbest* remains the same.

PSO Illustrated



- If its new position is better than *pbest*, then *pbest* is updated.

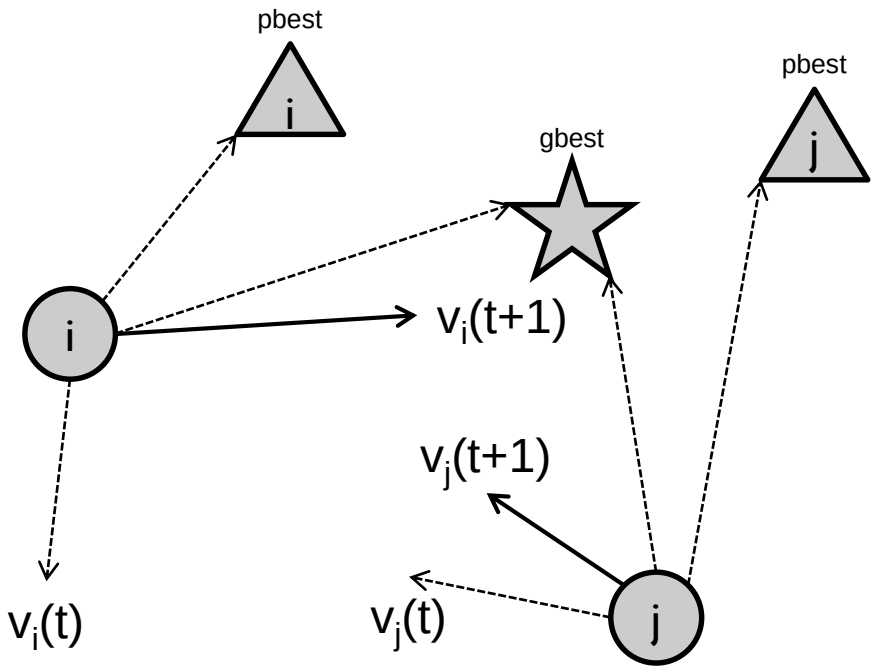
PSO Illustrated



- If its new position is better than *gbest*, then both *pbest* and *gbest* are updated.

PSO Illustrated

- This isn't happening in isolation – other particles (a whole **swarm** of them) are updating in parallel!



PSO Pseudocode

```
BEGIN
    INITIALISE population
    REPEAT UNTIL ( termination condition IS satisfied ) DO
        UPDATE global best;
        FOR EACH ( particle IN population ) DO
            1. UPDATE velocity and position;
            2. EVALUATE new position;
            3. UPDATE personal best;
        OD
    OD
END
```

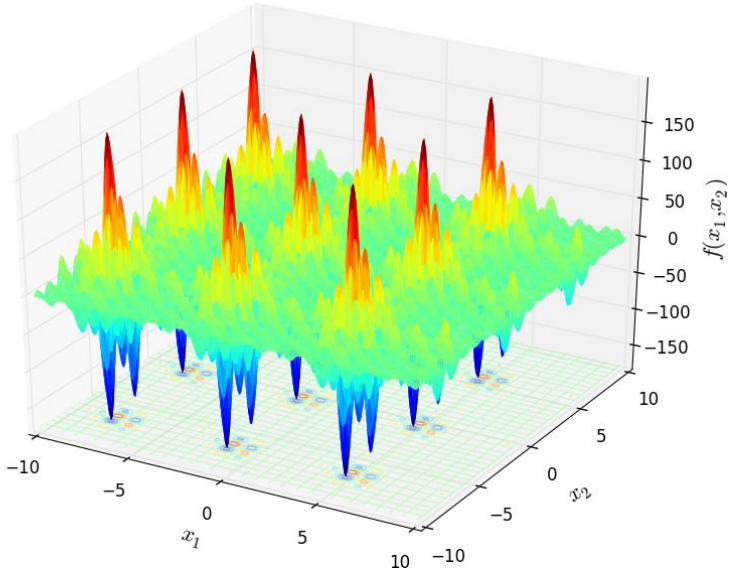
An Iteration

Particle Swarm Optimization

- Key insight: finding a roost base on local information equivalent to **real-valued optimization**.



=



- **Particle Swarm Optimization (PSO)** born!

PSO: Formalisation



PSO as Real Valued Optimizer

- Hopefully easy to see the connection between PSO and real-valued optimization:
  - **Positions** as candidate solutions
  - Choice of **pbest** and **gbest** based on cost function.
  - Update of **velocity** as strategy for traversing the search space.

- When **pbest** and **gbest** are very dissimilar, search is somewhat random.
- When **pbest** and **gbest** are very similar, search is somewhat like hill-climbing.
- Range of behaviours in between.



PSO Notation

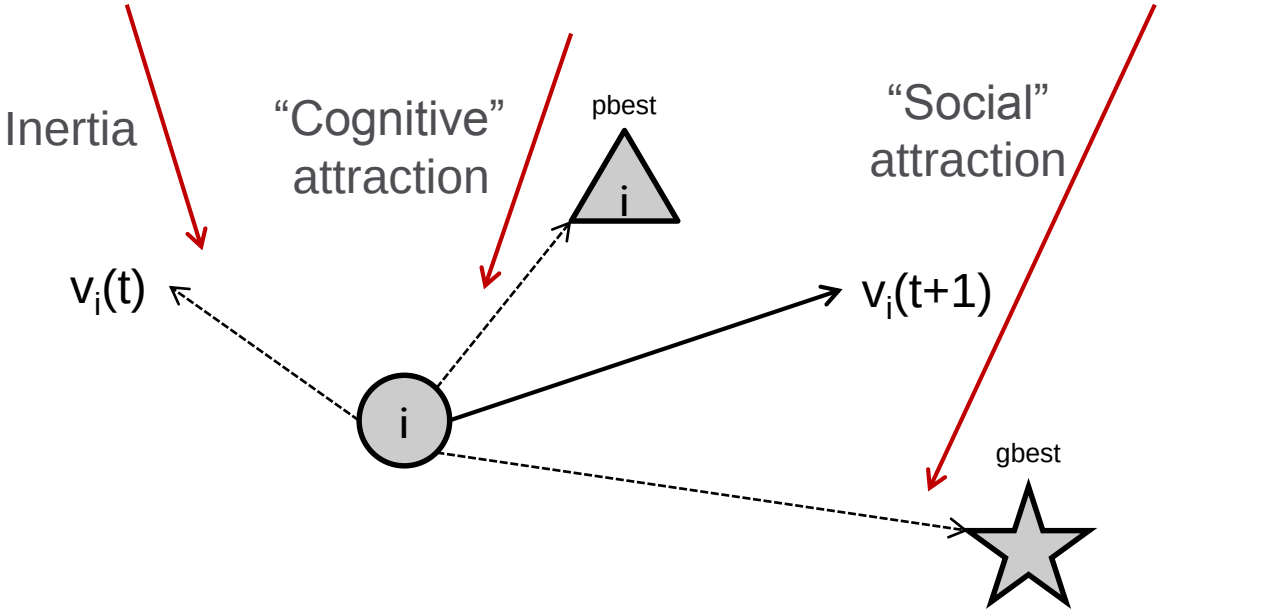
- Each particle i has:
  - $\overrightarrow{x_i}(t)$ – position
  - $\overrightarrow{v_i}(t)$ – velocity
  - $\overrightarrow{p_i}(t)$ – personal best position
- Swarm has:
  - $\vec{g}(t)$ – global best position
- Note:
  - t represents the **time**, which moves in steps of 1.
  - The arrow above $\overrightarrow{x_i}(t)$ says that it is a vector.



PSO: Velocity

$$\overrightarrow{x_i}(t+1) = \overrightarrow{x_i}(t) + \overrightarrow{v_i}(t+1)$$

$$\overrightarrow{v_i}(\mathrm{t}+1) = \eta \cdot \overrightarrow{v_i}(\mathrm{t}) + \phi_1 \cdot \overrightarrow{r_1} \cdot (\overrightarrow{p_i}(t) - \overrightarrow{x_i}(t)) + \phi_2 \cdot \overrightarrow{r_2} \cdot (\vec{g}(t) - \overrightarrow{x_i}(t))$$



$\vec{r}_1, \vec{r}_2 \sim U[0,1]^d$ - Uniform random vectors (regenerated every iteration).

η, ϕ_1, ϕ_2- "inertial", "cognitive" and "social" coefficients.

PSO Parameters

- Just like there is no one best algorithm, there is no one best parameter set.
- Some parameters known to have some good properties:
  - Swarm size: $20 + \sqrt{D}$ where D is problem dimension.
  - $\eta = \frac{1}{2 \ln 2} \simeq 0.721$
  - $\phi_1 = \phi_2 = \frac{1}{2} + \ln 2 \simeq 1.1193$
- Trelea proposes a parameter tuning heuristic (see recommended reading).

PSO: Initialisation

Variable	Initial Value
Position $\overrightarrow{x_i}(0)$	Randomly generated to lie in the **feasible region**.
Velocity $\overrightarrow{v_i}(0)$	A second feasible position is generated at random. Initial velocity is set to half the difference between this second position and $\overrightarrow{x_i}(0)$.
Personal Best Position $\overrightarrow{p_i}(0)$	Set to the initial position.



PSO: Constraint Handling

- Many ways proposed to deal with violation of constraints:
  - **Velocity clamping** – decrease the chance of escaping feasible region.
  - **Boundary "walls"** – prevent escape.
  - Solution **repair** – bring solutions back inside feasible region.

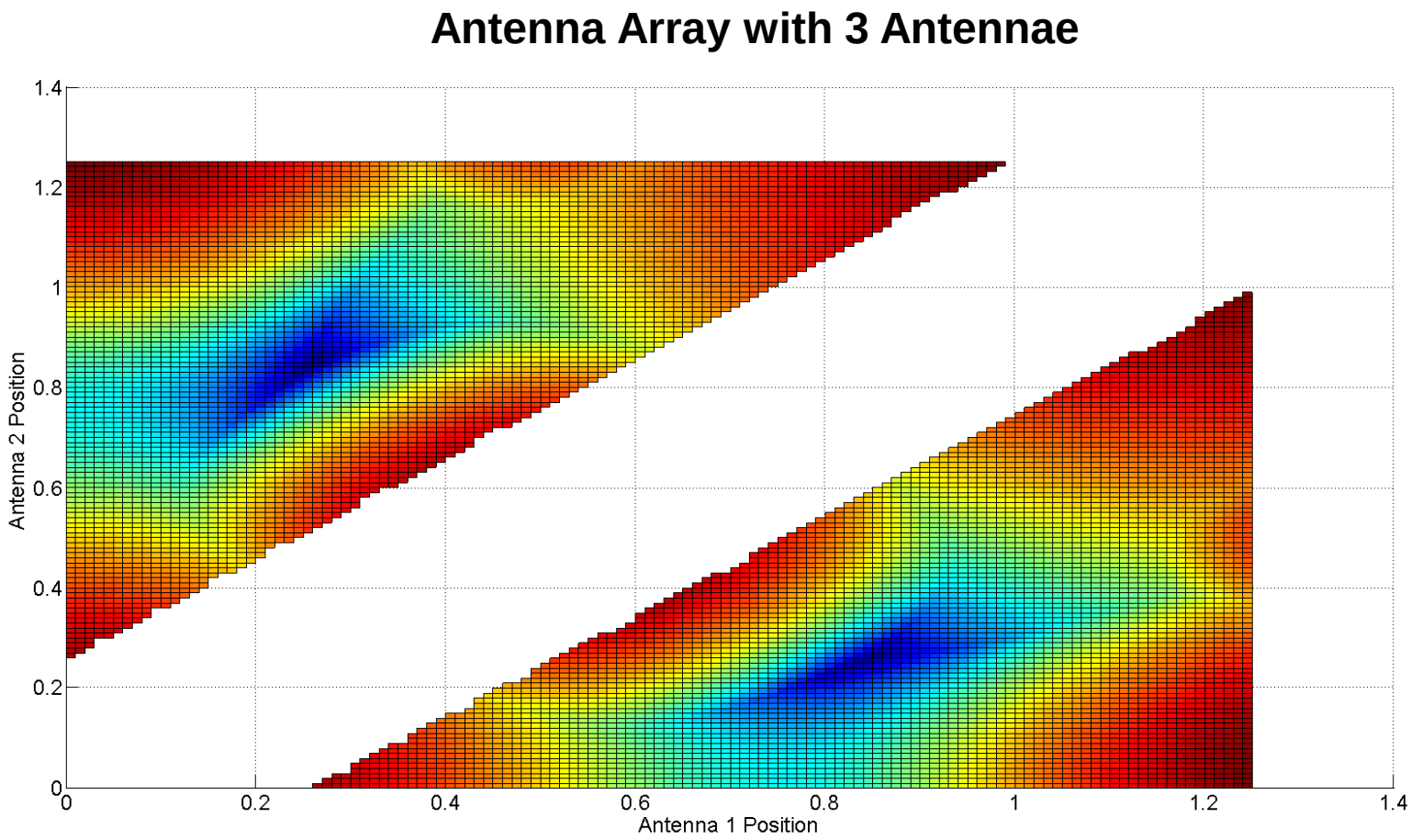
- Simple proposed method:
  - Check whether a particle's current position violates any constraints.
  - If it does, don't evaluate the solution.
  - Attraction to personal and global best will cause it to return to feasible region.
  - Called **invisible wall**.



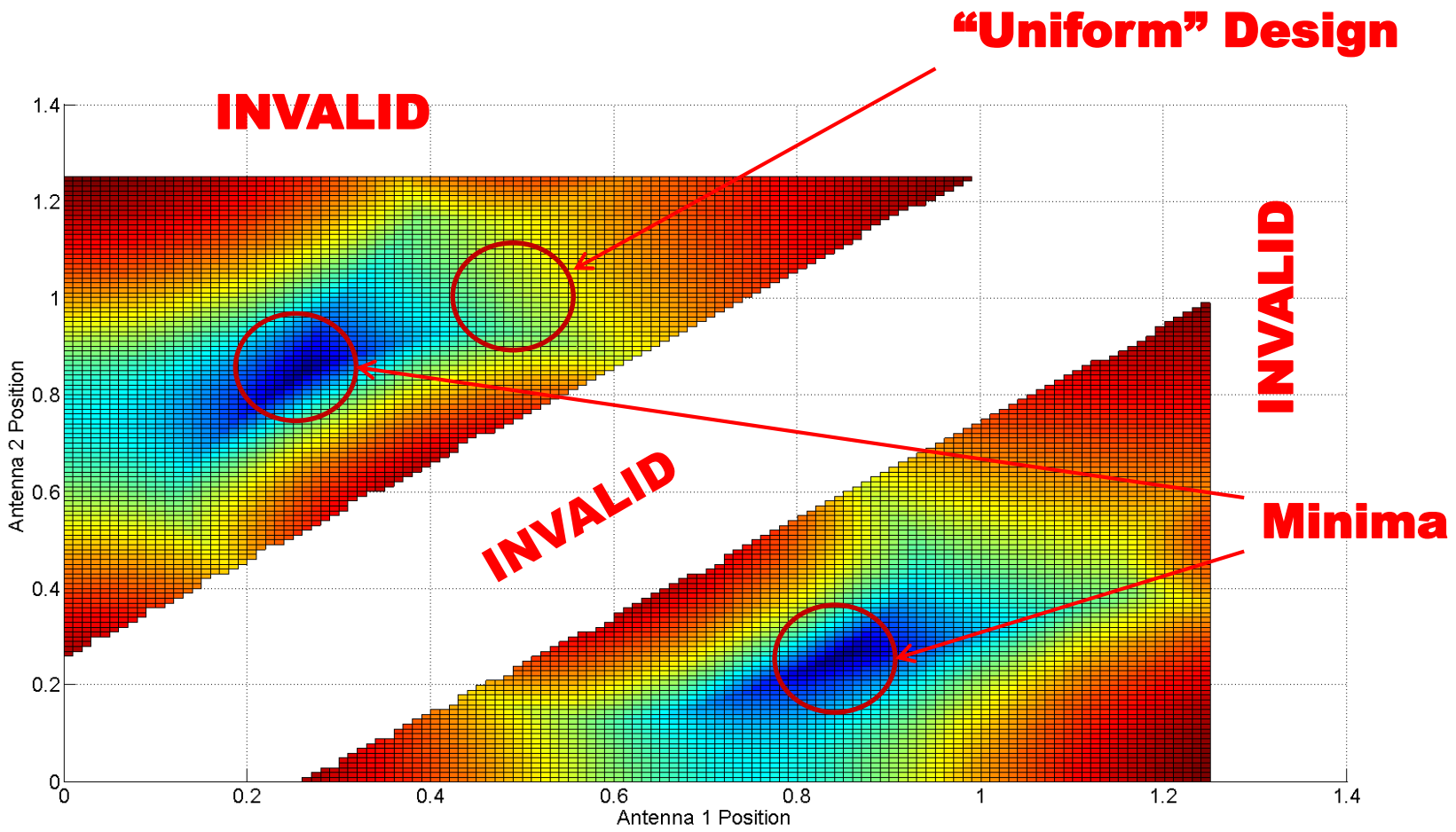
PSO: Application



Visualising the Problem



Visualising the Problem: Characteristics



Antenna Array Design with PSO

- You will be tackling this problem in the lab. Think about:

- How to generate a valid initial population?
  - Simple strategy?
    - Generate arbitrary solution at random
    - Check if it violates constraints
    - If not, accept it. If so, try again!

  - Not a bad solution for small number of antennae, but failure rate increases exponentially.

  - Can you think of something which always generates a valid solution? Do you need to!?



Antenna Array Design with PSO

- You will be tackling this problem in the lab. Think about:
- How to generate a valid initial population?
- How to guarantee valid solutions are generated during search?
  - Bounding the search space?
  - Reject invalid moves?
  - Do you need to!?



Antenna Array Design with PSO

- You will be tackling this problem in the lab. Think about:
- How to generate a valid initial population?
- How to guarantee valid solutions are generated during search?
- What to do if invalid solutions are generated
  - Invisible bounds?
  - Repair?



Antenna Array Design with PSO

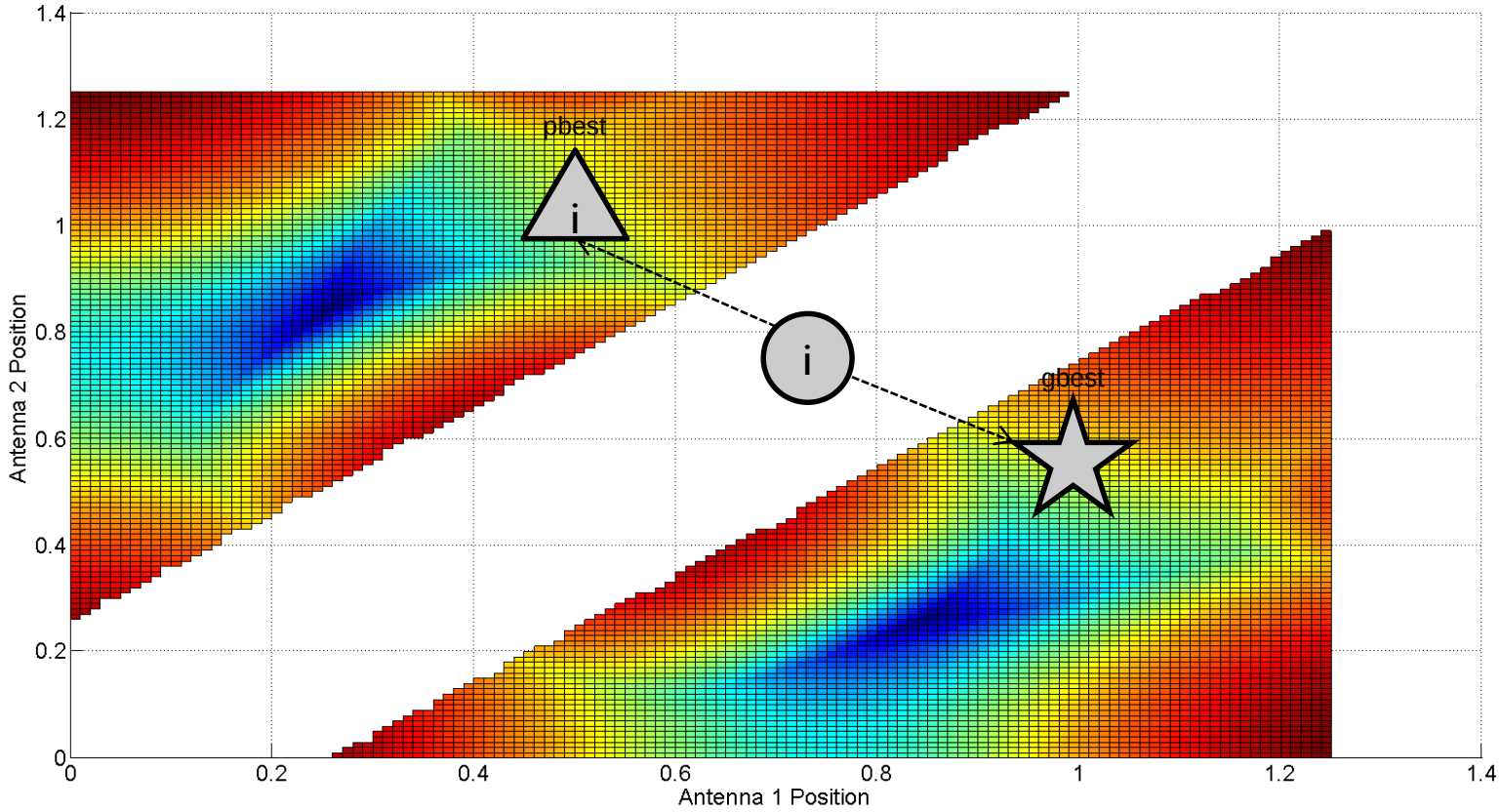
- We also need to consider properties of our algorithm.
- What is the difference between the following positions?

$$\overrightarrow{x_i}(t) = [0.5,1.0]$$
$$\overrightarrow{x_j}(t) = [1.0,0.5]$$

- Phenotypically: Nothing
- For PSO: ???



Antenna Array Design with PSO



Antenna Array Design with PSO

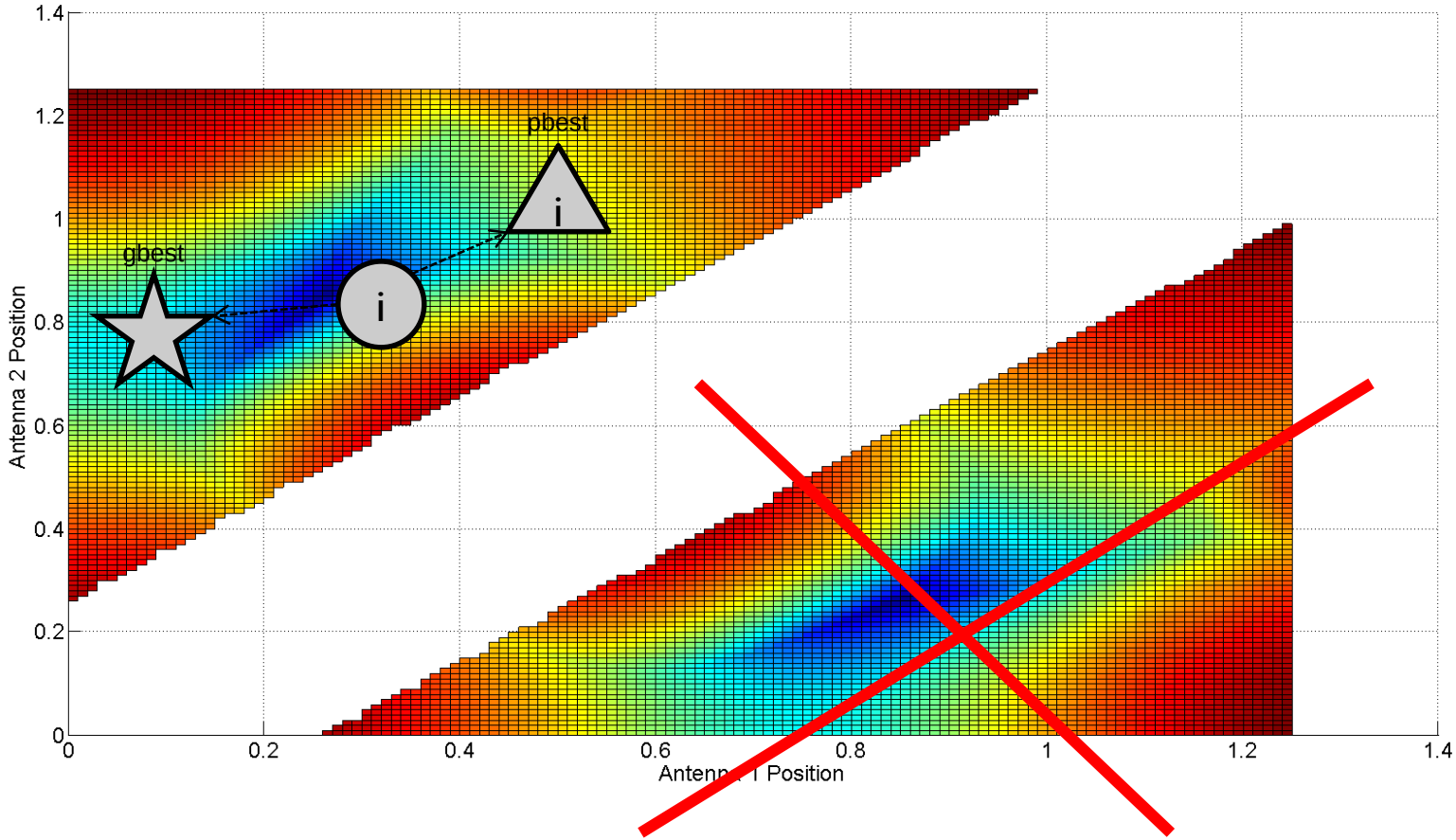
- We also need to consider properties of our algorithm.
- What is the difference between the following positions?

$$\overrightarrow{x_i}(t) = [0.5,1.0]$$
$$\overrightarrow{x_j}(t) = [1.0,0.5]$$

- Phenotypically: Nothing.
- For PSO:
  - Positions are either side of invalid region.
  - Can encourage the generation of invalid solutions.
  - Suggestion – add an extra constraint enforcing ordered solutions.



Antenna Array Design with PSO



Summary

- At the end of this lecture you should…
  - …be able to explain what Swarm Intelligence is and outline some key developments in its history.
  - …be able to describe the ideas behind the development of Particle Swarm Optimization.
  - …understand a given variant of Particle Swarm Optimization well enough to implement it and apply it to real valued optimization problems.



Recommended Reading

- Books
  - Bonabeau, Theraulaz and Dorigo: *Swarm Intelligence: From Natural to Artificial Systems*, Chapter 1
  - Eberhart, Kennedy and Shi: *Swarm Intelligence*, Chapter 7

- Papers
  - Trelea: *The particle swarm optimization algorithm: convergence analysis and parameter selection*
  - Jin and Rahmat-Samii: *Advances in Particle Swarm Optimization for Antenna Designs: Real-Number, Binary, Single-Objective and Multiobjective Implementations*
  - Poli, Kennedy and Blackwell: *Particle Swarm Optimization. An overview.*